



CHAPTER 01

জাভা প্রোগ্রামিং-এর ধারণা

Overview of Java

Introduction • History • Features • JVM • Applications
Programming Style • Conventions • Java Program Structure

Based on the uploaded BTEB-style chapter material

Md. Ruhul Amin • Junior Instructor, CST • Kushtia Government Polytechnic Institute

LEARN • CODE • BUILD

শিখনফল (Learning Outcomes)

এই অধ্যায় শেষে শিক্ষার্থীরা Java-এর ভিত্তি ও প্রোগ্রাম কাঠামো ব্যাখ্যা করতে পারবে।

01

Java কী ও কেন ব্যবহার করা হয় তা ব্যাখ্যা করা

02

Java-এর ইতিহাস ও বিকাশের প্রধান ধাপগুলো বর্ণনা করা

03

Java-এর প্রধান বৈশিষ্ট্যগুলো শনাক্ত ও ব্যাখ্যা করা

04

JVM, Bytecode ও Platform Independence-এর সম্পর্ক বোঝা

05

Java-এর বিভিন্ন ব্যবহারক্ষেত্র ও application style আলাদা করা

06

Programming style, naming convention ও comments অনুসরণ করা

07

একটি Java program-এর মৌলিক structure শনাক্ত করা

অধ্যায়ের রোডম্যাপ

একটি Java program বোঝার জন্য ধারণা → বৈশিষ্ট্য → পরিবেশ → প্রয়োগ → structure—এই flow অনুসরণ করা হবে।

01

Introduction

Java-এর প্রাথমিক ধারণা

02

History

Java কীভাবে তৈরি ও বিকশিত হলো

03

Features

Java-এর গুরুত্বপূর্ণ বৈশিষ্ট্য

04

Environment

JDK, path, compile & run

05

JVM

Bytecode থেকে machine code

06

Applications

Console, GUI, Applet

07

Conventions

Style, naming & comments

08

Structure

Java program-এর অংশগুলো

Java কী?

একটি object-oriented ও platform-independent programming language হিসেবে Java-এর পরিচিতি।

JAVA

Programming Language

Object-Oriented
Portable
Platform Independent

- জনপ্রিয় high-level programming language
- বিভিন্ন ধরনের software ও application তৈরিতে ব্যবহৃত
- Object-oriented design-এর ওপর জোর দেয়
- Bytecode ও JVM-এর মাধ্যমে platform independence অর্জন করে

OOP

Class • Object

JVM

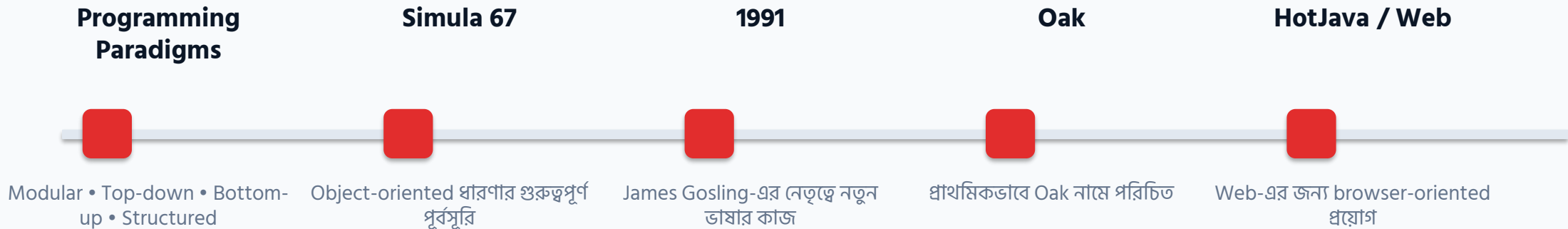
Bytecode • Runtime

Portable

Platform neutral

Java-এর ইতিহাস — সংক্ষিপ্ত টাইমলাইন

উৎস PDF-এর বর্ণনা অনুযায়ী Java-এর শুরুর দিকের গুরুত্বপূর্ণ ঘটনাগুলো।



পরবর্তী পর্যায়ে Java-এর version/JDK releases ধারাবাহিকভাবে প্রকাশিত হয়।

Java-এর জন্ম: Oak থেকে Java

উৎস অধ্যায়ে James Gosling, Oak এবং Web-oriented HotJava-এর উল্লেখ রয়েছে।

James Gosling

1991 সালে Sun Microsystems-এ নতুন programming language development-এর নেতৃত্বে ছিলেন।

Oak

ভাষাটির প্রাথমিক নাম ছিল Oak। পরে Java নাম গ্রহণ করা হয়।

HotJava

Web browser ও applet-কেন্দ্রিক প্রয়োগের সঙ্গে Java-এর বিস্তার ঘটে।



Java Version / JDK বিবর্তনের ধারণা

উৎস অধ্যায়ে JDK 1.01, 1.02, 1.1, 1.2, 1.3, 1.4, 1.5 এবং Java 2-এর মতো releases-এর উল্লেখ আছে।

JDK 1.01

প্রাথমিক development kit

JDK 1.1

ক্রমাগত উন্নয়ন

JDK 1.2

Java 2 নামের ব্যবহার

JDK 1.3–1.5

পরবর্তী ধারাবাহিক release

মূল শিক্ষা: Java ecosystem-এর development kit ও runtime tooling সময়ের সঙ্গে ধাপে ধাপে বিকশিত হয়েছে।

Java-এর বৈশিষ্ট্য — এক নজরে

উৎস PDF-এর তালিকা অনুসারে প্রধান বৈশিষ্ট্যগুলো।

Object Oriented

Object-based design

Platform Independent

JVM-এর মাধ্যমে

Highly Efficient

দক্ষ execution

Robust & Secure

নিরাপদ ও নির্ভরযোগ্য

Distributed

network-oriented use

Compiled + Interpreted

দুই-ধাপের execution

Portable

সহজে platform বদলানো

Simple / Small

সহজ, সংক্ষিপ্ত

Dynamic & Extensible

runtime-এ সম্প্রসারণ

Multithreaded

একাধিক কাজ একসঙ্গে

Object Oriented

Java একটি object-oriented language; class, object এবং OOP principles-এর ওপর ভিত্তি করে design করা যায়।

Object Oriented

Java একটি object-oriented language; class, object এবং OOP principles-এর ওপর ভিত্তি করে design করা যায়।

KEY IDEA

01 **Class**

02 **Object**

03 **Inheritance**

04 **Polymorphism**

05 **Encapsulation**

Both Compiled & Interpreted

Java source code প্রথমে compiler দ্বারা bytecode-এ রূপান্তরিত হয়; পরে JVM-side interpreter/processing-এর মাধ্যমে machine execution-এর দিকে যায়।

Both Compiled & Interpreted

Java source code প্রথমে compiler দ্বারা bytecode-এ রূপান্তরিত হয়; পরে JVM-side interpreter/processing-এর মাধ্যমে machine execution-এর দিকে যায়।

KEY IDEA

01 Source Code

02 `javac`

03 Bytecode

04 JVM

05 Machine Code

Platform Independent

একই bytecode বিভিন্ন platform-এর জন্য সংশ্লিষ্ট JVM/Interpreter ব্যবহার করে চালানো যায়—এটাই 'write once, run on different platforms' ধারণার ভিত্তি।

Platform Independent

একই bytecode বিভিন্ন platform-এর জন্য সংশ্লিষ্ট JVM/Interpreter ব্যবহার করে চালানো যায়—এটাই 'write once, run on different platforms' ধারণার ভিত্তি।

KEY IDEA

01 **Windows**

02 **Linux / Unix**

03 **Solaris**

04 **Mac**

Portable

Java program ও bytecode নির্দিষ্ট machine architecture-এর ওপর সরাসরি নির্ভরশীল না হওয়ায় এক platform থেকে অন্য platform-এ নেওয়া তুলনামূলক সহজ।

Portable

Java program ও bytecode নির্দিষ্ট machine architecture-এর ওপর সরাসরি নির্ভরশীল না হওয়ায় এক platform থেকে অন্য platform-এ নেওয়া তুলনামূলক সহজ।

KEY IDEA

01 **Bytecode**

02 **JVM**

03 **Portability**

Highly Efficient

উৎস অধ্যায় Java-এর efficiency-কে compiled/interpreted execution এবং runtime environment-এর সঙ্গে যুক্ত করেছে।

Highly Efficient

উৎস অধ্যায় Java-এর efficiency-কে compiled/interpreted execution এবং runtime environment-এর সঙ্গে যুক্ত করেছে।

KEY IDEA

01 **Compile**

02 **Bytecode**

03 **Runtime**

Simple, Familiar & Small

C/C++-এর সঙ্গে পরিচিত syntax-এর অনেক মিল থাকায় শেখা সহজ; ভাষাটিকে তুলনামূলক simple ও small হিসেবে উপস্থাপন করা হয়েছে।

Simple, Familiar & Small

C/C++-এর সঙ্গে পরিচিত syntax-এর অনেক মিল থাকায় শেখা সহজ; ভাষাটিকে তুলনামূলক simple ও small হিসেবে উপস্থাপন করা হয়েছে।

KEY IDEA

01 **Familiar syntax**

02 **Simple structure**

03 **Small language**

Robust & Secure

Exception handling, runtime checks এবং security-related design-এর মাধ্যমে Java-কে robust ও secure হিসেবে বর্ণনা করা হয়েছে।

Robust & Secure

Exception handling, runtime checks এবং security-related design-এর মাধ্যমে Java-কে robust ও secure হিসেবে বর্ণনা করা হয়েছে।

KEY IDEA

01 **Reliability**

02 **Security**

03 **Exception handling**

Dynamic & Extensible

Runtime-এ class/object-এর সঙ্গে কাজ ও program-কে প্রয়োজন অনুযায়ী সম্প্রসারণের ধারণা Java-এর dynamic nature-এর সঙ্গে যুক্ত।

Dynamic & Extensible

Runtime-এ class/object-এর সঙ্গে কাজ ও program-কে প্রয়োজন অনুযায়ী সম্প্রসারণের ধারণা Java-এর dynamic nature-এর সঙ্গে যুক্ত।

KEY IDEA

01 **Dynamic loading**

02 **Classes**

03 **Extensible**

Distributed

Network environment-এ কাজ করার জন্য Java-এর networking support ও distributed application ধারণা উল্লেখ করা হয়েছে।

Distributed

Network environment-এ কাজ করার জন্য Java-এর networking support ও distributed application ধারণা উল্লেখ করা হয়েছে।

KEY IDEA

01 **Network**

02 **Sockets**

03 **Distributed objects**

Multithreaded

একই application-এ একাধিক কাজ/থ্রেড সমান্তরালভাবে চালানোর সুবিধাকে Java-এর multithreaded বৈশিষ্ট্য হিসেবে দেখানো হয়েছে।

Multithreaded

একই application-এ একাধিক কাজ/থ্রেড সমান্তরালভাবে চালানোর সুবিধাকে Java-এর multithreaded বৈশিষ্ট্য হিসেবে দেখানো হয়েছে।

KEY IDEA

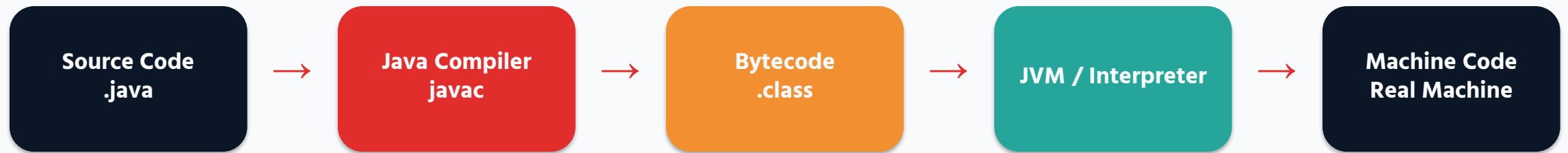
01 Thread 1

02 Thread 2

03 Thread 3

Java Virtual Machine (JVM)

Java source → bytecode → JVM → machine code: portability-এর মূল ধারণা।



Phase 1

Source code → Bytecode

Phase 2

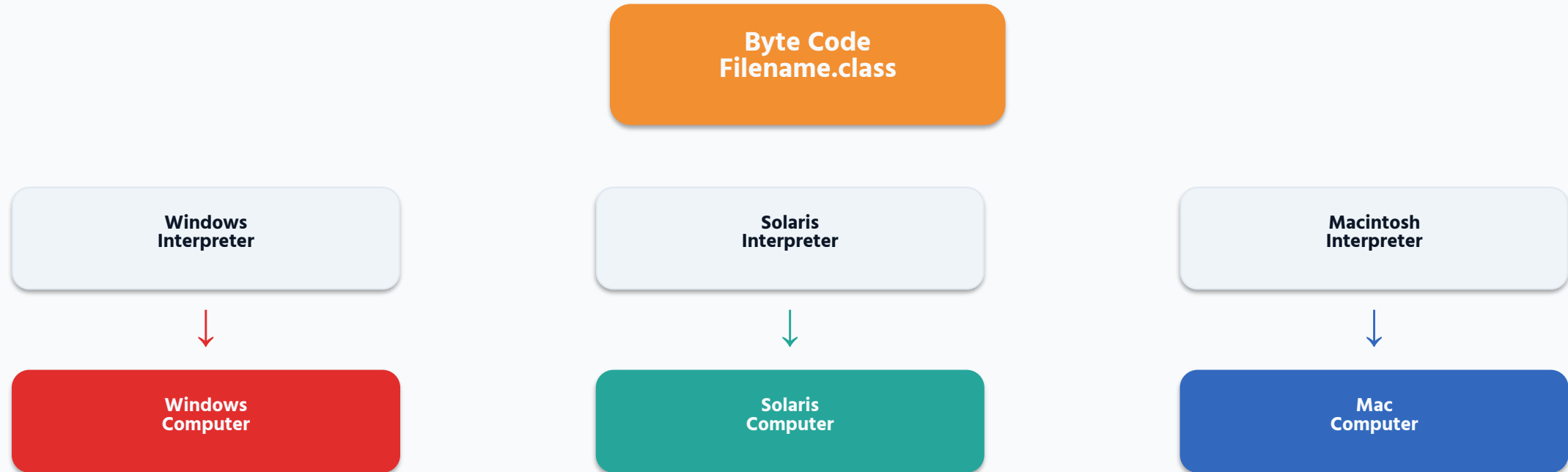
Bytecode → JVM execution

Result

Platform-specific machine execution

Platform Independence — একই Bytecode, ভিন্ন Platform

উৎস PDF-এর diagram-এর ধারণা: একই .class bytecode বিভিন্ন platform-এর interpreter/JVM দিয়ে চালানো যায়।



Key takeaway: Platform-specific অংশটি runtime environment-এ handled হয়।

Windows-এ Java Environment Setup

উৎস অধ্যায়ে environment setup-এর চারটি ধাপ দেখানো হয়েছে।



01

JDK Installation

Java Development Kit install

02

Path Setting

DOS prompt-এ path set

03

Program Coding

Text editor-এ source code

04

Compile & Run

javac দিয়ে compile, java দিয়ে run

Setup flow

JDK → PATH → Coding → javac → java

JDK — Java Development Kit

Java development-এর জন্য compiler, interpreter/runtime এবং documentation-related tools-এর সমষ্টিগত environment।

javac

Java compiler

Source code → Bytecode

java

Java interpreter

Bytecode execution

javadoc

Documentation tool

Source থেকে HTML documentation

javah

Native header tool

Native method-এর জন্য header

javap

Bytecode tool

Bytecode/class information

jdb

Debugger

Java debugging

Path Setting ও Command Line Workflow

উৎস PDF-এ DOS prompt-এ PATH set করার উদাহরণ এবং compile/run command দেখানো হয়েছে।

C:\jdk 1.3\bin

PATH-এর মধ্যে Java executable-এর অবস্থান যুক্ত করা

C:\jdk 1.3\bin> javac start.java

Compile

C:\jdk 1.3\bin> java start

Run

Java-এর ব্যবহারক্ষেত্র (Applications)

উৎস অধ্যায়ে console, GUI এবং applet-সহ বিভিন্ন application context-এর কথা বলা হয়েছে।

Console Mode

Text-based interaction; সাধারণত command/terminal-এ কাজ করে।

GUI Application

Graphical User Interface; Button, TextField ইত্যাদি component ব্যবহার করা যায়।

Applet

HTML page-এর embedded environment-এ Java program চালানোর ঐতিহাসিক পদ্ধতি।

Web / Network

Network-oriented ও distributed application-এর জন্য Java ব্যবহারের ধারণা।

শিক্ষণ টিপ: প্রথমে **Console** → তারপর **GUI** → তারপর **network/distributed** ধারণায় এগোনো সহজ।

Console বনাম GUI Application

দুই ধরনের application style-এর মৌলিক পার্থক্য।

Console Mode

- Text-based input/output
- Keyboard/terminal-কেন্দ্রিক
- সাধারণত graphical component নেই
- Program output দ্রুত পরীক্ষা করা যায়

Console

GUI Application

- Graphical interface
- Window, Button, TextField ইত্যাদি
- User interaction visual
- Interface-এর মাধ্যমে application control

GUI

Java Programming Style & Convention

Readable, consistent এবং maintainable code লেখার জন্য naming ও indentation convention গুরুত্বপূর্ণ।

Class Naming

প্রথম অক্ষর Capital; একাধিক শব্দ হলে প্রতিটি শব্দের শুরু Capital.

Example:
CapitalizedEveryWord
MyClass

Constant Naming

Capital letters এবং প্রয়োজনে underscore ব্যবহার।

Example:
CAPITALIZE_WITH_UNDERSCORE
MY_CONSTANT

Method / Field

সাধারণত প্রথম শব্দ lowercase; পরের শব্দের শুরু Capital.

Example:
startWithLowercaseAndCapitalizeOtherWords
myMethod
myField

Java Comments

Program-এর অংশকে ব্যাখ্যা, document বা temporarily exclude করার জন্য comments ব্যবহার করা হয়।

```
// This is a single line comment
```

```
sum = sum + x; // This is a single line comment
```

```
/* This program is used to find out  
the summation of a series */
```

//

Single-line comment

/* */

Multi-line comment

Why?

Readability & documentation

Java Program-এর মৌলিক Structure

উৎস অধ্যায়ের structure diagram অনুযায়ী একটি Java program-এ কয়েকটি section থাকতে পারে; কিছু section optional।

Documentation section



Package statements



Import statements



Class definition



Inheritance statements



Interface statements



main class definition



main() method definition



Other methods calling

Program Flow

Documentation
↓
Package / Import
↓
Class
↓
main()
↓
Other methods

একটি Basic Java Program

উৎস অধ্যায়ে Myfirstjava ধরনের sample structure দিয়ে class, main method ও System.out.println দেখানো হয়েছে।

```
public class Myfirstjava
{
    public static void main(String args[])
    {
        System.out.println("I love Bangladesh");
    }
}
```

class

Class declaration / class body

main()

Program entry point

println()

Output statement

Program Structure-এর গুরুত্বপূর্ণ অংশ

উৎস অধ্যায়ের package, import, inheritance এবং interface statement-এর উদাহরণ।

Package

```
package mypackage;
```

Program-এ classes/interfaces organize করার জন্য।

Import

```
import java.awt.*;
```

অন্য package-এর class ব্যবহার করার জন্য।

Inheritance

```
class A extends B
```

এক class থেকে অন্য class-এ inheritance।

Interface

```
class A extends B implements C
```

Interface implement করার syntax।

main() Method — কেন গুরুত্বপূর্ণ?

উৎস অধ্যায় main class ও main() method-কে Java program-এর execution context-এর সঙ্গে যুক্ত করেছে।

```
public static void main  
(String args[])
```

Program entry point

public

Access visibility

static

Class-level method

void

No return value

main

Method name

String args[]

Command-line arguments

অধ্যায় ১ — Quick Recap

পরবর্তী অধ্যায়ে যাওয়ার আগে এই ৮টি বিষয় পরিকল্পনা থাকা উচিত।

01

Java পরিচিতি

02

Java-এর ইতিহাস

03

10টি প্রধান বৈশিষ্ট্য

04

Bytecode ও JVM

05

Platform independence

06

JDK / PATH / Compile / Run

07

Programming convention

08

Java program structure

Concept → Code → Compile → Run → Understand

Classroom Practice / Discussion

শিক্ষার্থীদের দিয়ে ধারণাগুলো active recall করানো যেতে পারে।

1. Java-কে platform independent বলা হয় কেন?

2. Source code ও bytecode-এর পার্থক্য কী?

3. JVM-এর ভূমিকা কী?

4. Java-এর পাঁচটি feature লিখো।

5. Console ও GUI application-এর পার্থক্য বলো।

6. javac এবং java command-এর কাজ কী?

7. Class naming ও constant naming convention কী?

8. একটি Java program-এর প্রধান sectionগুলো লেখো।

ধন্যবাদ

Java Programming • Chapter 01

Learn the concept.
Understand the flow.
Then write the code.

Based on the uploaded chapter material